

CMPUT 267: Machine Learning I · Fall 2026

Supervised Learning and the ML Pipeline

Xi Ye

2026-09-15

Recall: Data, Learner, Predictor



Last lecture:

- each feature-label pair $(\mathbf{X}_i, Y_i)$ is random
- the dataset D is random
- therefore the learned predictor $\hat{f} = \mathcal{A}(D)$ is random.

Today: closer look at **supervised learning**.

Running Example: Predict a House Price

For each previously sold house, we observe:

Length (m)	Width (m)	Age (years)	Price (\$1000s)
12	8	25	310
15	10	8	510
11.5	10	40	355

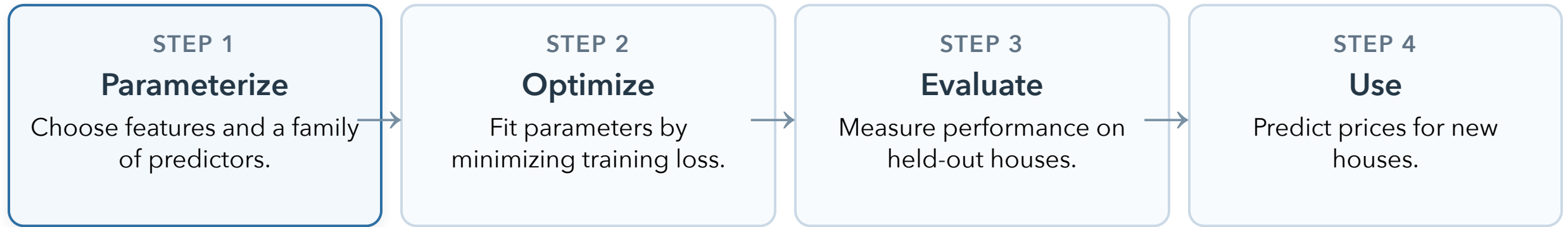
We want a predictor that maps information about a new house to its price.

$\mathcal{X}$ (input feature): (length, width, age)

$\mathcal{Y}$ (label): price

$$f : \mathcal{X} \rightarrow \mathcal{Y}, \quad f(\mathbf{x}) = \hat{y}.$$

A Practical ML Pipeline



This is a first practical map. We will make each step more precise over the next several lectures.

Parameterization: Linear Functions

Let $\mathbf{x} = (\ell, w, a)$ contain a house's length, width, and age. Consider the linear function

$$f_{\theta}(\mathbf{x}) = b + \theta_{\ell}\ell + \theta_w w + \theta_a a, \quad \theta = (b, \theta_{\ell}, \theta_w, \theta_a).$$

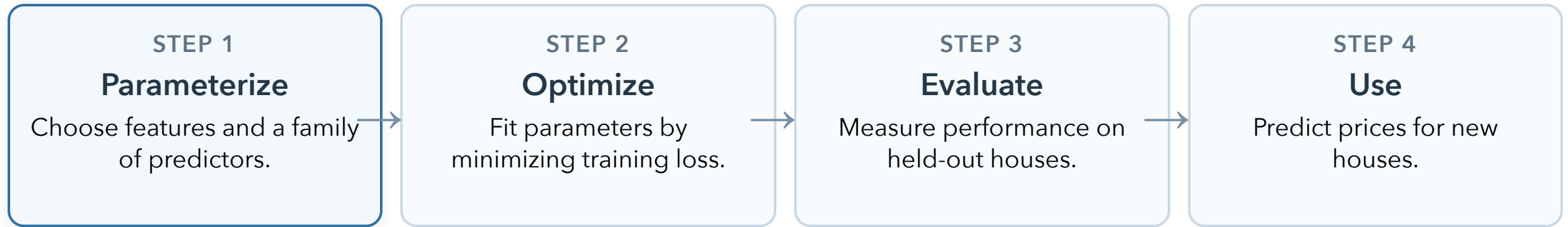
Given one parameter vector θ and one new house $\mathbf{x}$:

$$\theta = (50, 20, 15, -1), \quad \mathbf{x} = (12, 8, 20).$$

$$f_{\theta}(12, 8, 20) = 50 + 20(12) + 15(8) - 1(20) = 390.$$

The predicted price is \$390,000.

Parameterization: Choices



 A parameterization chooses the features a predictor uses and the shapes of functions it can represent.

Parameterization Choice: Function Shape

Suppose we also record a house's height h . We predict price from age a and height h using either family.

Simple: Linear

$$f_{\theta}(a, h) = \theta_0 + \theta_a a + \theta_h h$$

Price changes at a constant rate with each feature.

Flexible: Polynomial

$$f_{\theta}(a, h) = \theta_0 + \theta_a a + \theta_h h + \theta_{aa} a^2 + \theta_{ah} ah + \theta_{hh} h^2$$

Squared and interaction terms allow the relationship to curve.

$$\mathcal{F}_{\text{linear}} \subset \mathcal{F}_{\text{quadratic}}.$$

Both families use the same house features. The polynomial family allows more flexible prediction functions.

Parameterization Choice: Construct a Feature

Length (m)	Width (m)	Age (years)	Price (\$1000s)
12	8	25	310
15	10	8	510
11.5	10	40	355

One can **create new features** by combining existing ones. For instance, length and width can be combined into **floor area**: $\text{area} = \ell w$.

The augmented family is

$$f_{\theta}(\mathbf{x}) = b + \theta_{\ell}\ell + \theta_w w + \theta_a a + \theta_{\text{area}}(\ell w).$$

From Parameterization to Learning

Parameterization specifies the candidate predictors:

$$\mathcal{F} = \{f_{\theta} : \theta \in \Theta\}.$$

But it does not tell us which parameter value to choose.

② How can we compare two predictors on the same house?

We need a numerical measure of prediction quality: a **loss function**.

Loss Function

A **loss function** assigns a nonnegative cost to one prediction and its true label. Smaller loss is better.

For house price prediction, a regression task, a common choice is squared loss:

$$\ell(\hat{y}, y) = (\hat{y} - y)^2.$$

If the true price is $y = 400$:

Prediction

$$\hat{y} = 390$$

$$\text{Loss} = (-10)^2 = 100$$

Prediction

$$\hat{y} = 350$$

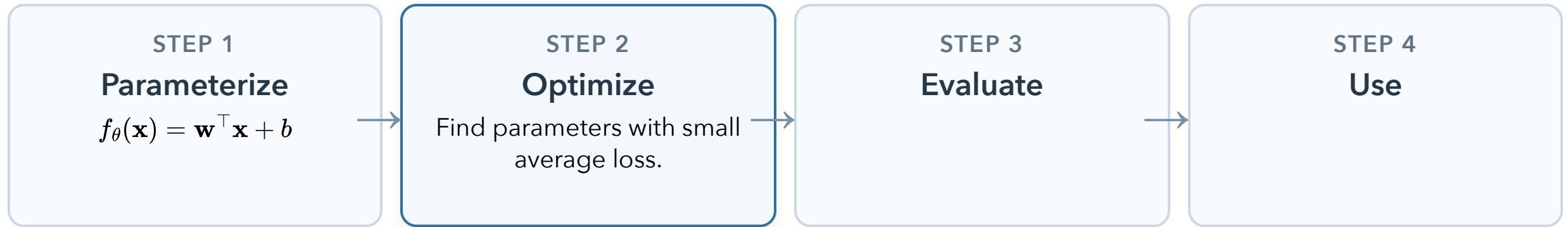
$$\text{Loss} = (-50)^2 = 2500$$

Prediction

$$\hat{y} = 500$$

$$\text{Loss} = (100)^2 = 10,000$$

Optimization Fits the Parameters



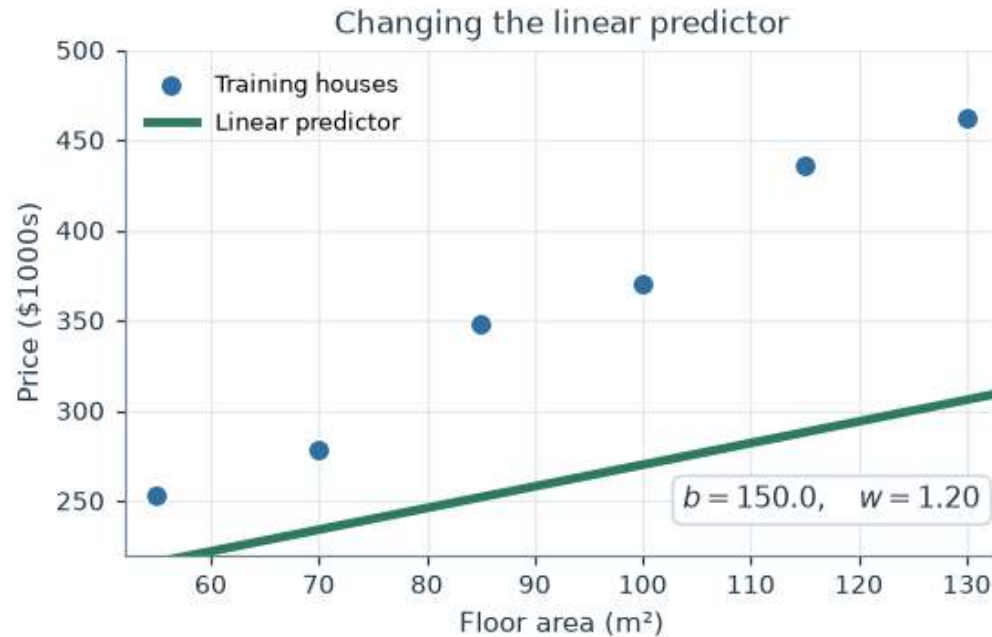
For a fixed training dataset $\mathcal{D}_{\text{train}}$:

$$\hat{\theta} \in \arg \min_{\theta} \frac{1}{n_{\text{train}}} \sum_{i \in \text{train}} \ell(f_{\theta}(\mathbf{x}_i), y_i).$$

i For now, optimization is the procedure that searches for $\hat{\theta}$. We study how it searches later.

What Optimization Changes

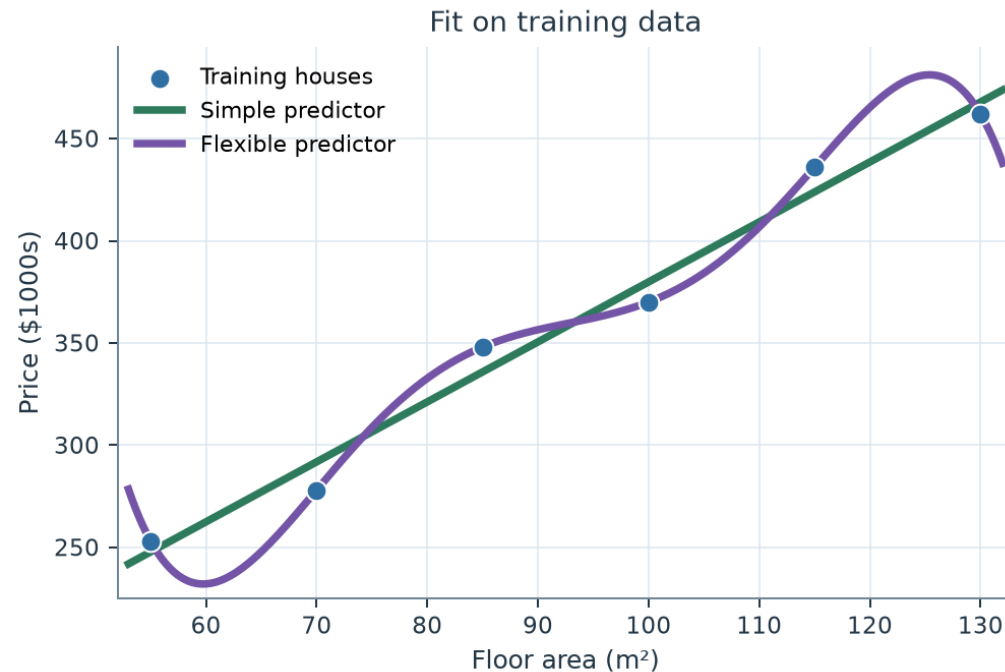
Each $\theta = (b, w)$ gives a different linear predictor $f_{\theta}(s) = b + ws$.



 Optimization changes the parameters and compares the resulting training loss. We will study the search procedure later.

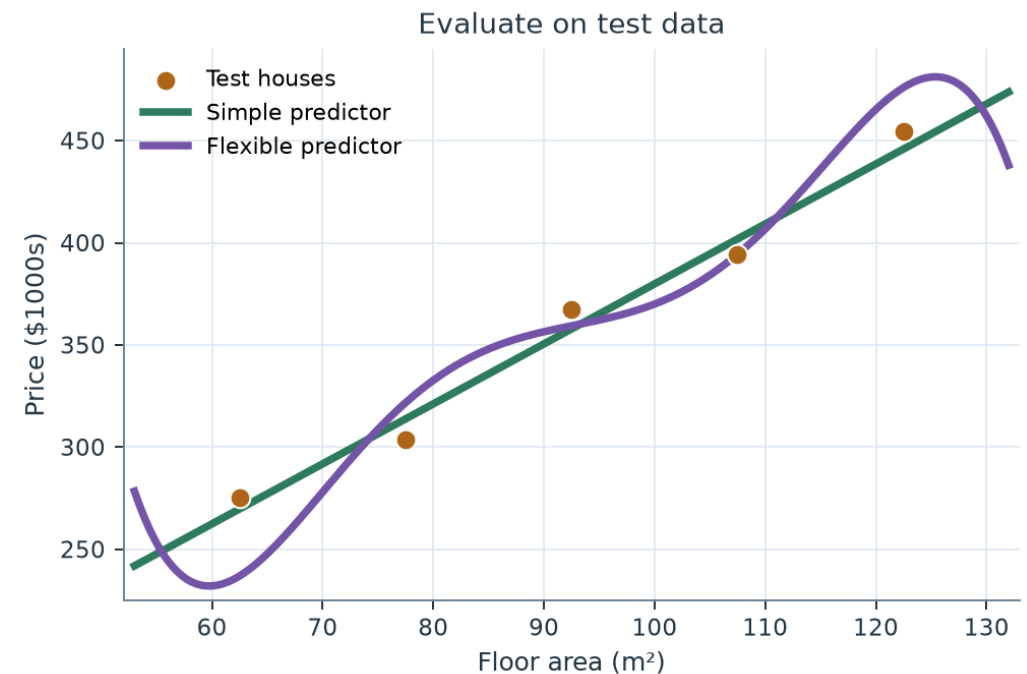
Training and Test Loss Can Rank Predictors Differently

We fit on the training houses (blue).



Training loss (MSE): simple 107.5, flexible 0.0

We test on new, unseen houses (orange).



Test loss (MSE): simple 72.1, flexible 469.3

Why We Need Held-Out Evaluation

If we compare only training loss, we choose the flexible predictor:

$$0.0 < 107.5.$$

But test loss reverses the comparison:

$$72.1 < 469.3.$$



Training data help **optimize** a predictor. Untouched test data help **evaluate** the optimized predictor on new houses.

We will study this selection effect rigorously later.

Training and Evaluation

Training

Use observed labels to optimize (learn) the predictor.

$$\mathcal{D}_{\text{train}} \longrightarrow \hat{f}$$

Evaluation

Use untouched labels to estimate how the chosen predictor performs.

$$\mathcal{D}_{\text{test}} \longrightarrow \hat{L}_{\text{test}}(\hat{f})$$



Evaluation is an **estimation problem**: use finite data to estimate unknown future performance.

Defining Supervised Learning

Supervised Learning



$$D = ((\mathbf{X}_1, Y_1), \dots, (\mathbf{X}_n, Y_n)) \in (\mathcal{X} \times \mathcal{Y})^n.$$

where

$$(\mathbf{X}_i, Y_i) \stackrel{\text{iid}}{\sim} P_{\mathbf{X}, Y}.$$

A **learner** $\mathcal{A}$ takes an entire dataset D and returns a **predictor** $\hat{f}$.

Training and Future Examples

The training pairs and a future feature-label pair follow the same distribution:

$$(\mathbf{X}_i, Y_i) \stackrel{\text{iid}}{\sim} P_{\mathbf{X}, Y}, \quad (\mathbf{X}, Y) \sim P_{\mathbf{X}, Y}.$$

For house-price prediction, the sold houses should represent the houses on which we will use the predictor.

 If the future population changes, the predictor faces **distribution shift**. Our standard supervised-learning objective assumes no such shift.

Learning Objective: Informal

Learner $\mathcal{A}$

Uses the sold houses to choose a prediction rule.

$$\mathcal{A} : D \mapsto \hat{f}$$

Predictor $\hat{f}$

Uses one new house's features to predict its price.

$$\hat{f} : \mathcal{X} \rightarrow \mathcal{Y}$$

$$\mathcal{A}(D) = \hat{f}.$$

Informal Objective (of Machine Learning):

Define a learner $\mathcal{A}$ such that its predictor $\hat{f}$ is **good**.

But we have not yet defined what “good” means.

Learning Objective: More Formal

For one feature $\mathbf{X}$, the predictor f incurs the loss

$$\ell(\hat{f}(\mathbf{X}), Y).$$

Risk is loss averaged over future houses according to $P_{\mathbf{X},Y}$:

$$L(\hat{f}) = \mathbb{E}_{(\mathbf{X},Y) \sim P_{\mathbf{X},Y}} [\ell(\hat{f}(\mathbf{X}), Y)].$$

Almost-formal objective. Define $\mathcal{A}$ such that $L(\hat{f})$ is small, where $\hat{f} = \mathcal{A}(D)$.

Learning Objective: Formal

Different training datasets can produce different predictors:

D is random $\longrightarrow \hat{f} = \mathcal{A}(D)$ is random $\longrightarrow L(\hat{f})$ is random

We therefore assess the learner by averaging risk **over possible training datasets**.

Objective: Define $\mathcal{A}$ such that $\mathbb{E}_D[L(\mathcal{A}(D))]$ is small.

Inner Expectation

$L(\mathcal{A}(D))$ averages over future feature-label pairs for one trained predictor.

Outer Expectation

$\mathbb{E}_D$ averages over the random training datasets we might have received.

Learning Objectives

Informal Objective (of Machine Learning):

Define a learner $\mathcal{A}$ such that its predictor $\hat{f}$ is **good**.

We then define what we mean by “good”.

Almost-formal objective. Define $\mathcal{A}$ such that $L(\hat{f})$ is small, where $\hat{f} = \mathcal{A}(D)$.

Since D is random, we ask for expected loss over all possible training datasets.

Final Objective: Define $\mathcal{A}$ such that $\mathbb{E}_D[L(\mathcal{A}(D))]$ is small.



In this course, when we study one observed, fixed dataset, we write $\mathcal{D}$ instead of the random variable D .

The Ideal Learning Objective

Given the true distribution, we would compare **every possible prediction function** $f : \mathcal{X} \rightarrow \mathcal{Y}$.

$$L(f) = \mathbb{E}_{(\mathbf{X}, Y) \sim P_{\mathbf{X}, Y}} [\ell(f(\mathbf{X}), Y)],$$
$$f^* \in \arg \min_{f: \mathcal{X} \rightarrow \mathcal{Y}} L(f).$$

This ideal plan has two obstacles: we do not know $P_{\mathbf{X}, Y}$, and searching every possible function is computationally infeasible.

Practical Choice 1: Empirical Risk

We only observe a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$. Use its average loss to estimate $L(f)$.

$$\underbrace{L(f)}_{\text{unknown true risk}} \longrightarrow \underbrace{\hat{L}_{\mathcal{D}}(f) = \frac{1}{n} \sum_{i=1}^n \ell(f(\mathbf{x}_i), y_i)}_{\text{empirical risk}}.$$

ERM with All Functions

Using $\mathcal{F}_{\text{all}}$, **empirical risk minimization (ERM)** has two steps.

Step 1: Estimation

Use $\mathcal{D}$ to estimate $L(f)$ for every $f \in \mathcal{F}_{\text{all}}$.

Step 2: Optimization

Choose $\hat{f} \in \mathcal{F}_{\text{all}}$ that minimizes the empirical risk.

$$\hat{f} \in \arg \min_{f \in \mathcal{F}_{\text{all}}} \hat{L}_{\mathcal{D}}(f).$$

ERM with Unrestricted Functions

We cannot enumerate and search every mapping from $\mathcal{X}$ to $\mathcal{Y}$ 😓

A function can memorize $\mathcal{D}$ and behave arbitrarily on inputs that were not observed. 😏



We restrict the set of predictors to a class of functions $\mathcal{F}$ which is just a set of predictors that is a strict subset of all possible predictors $\mathcal{F}_{\text{all}} = \{f | f : X \rightarrow Y\}$.

No Free Lunch Theorem (Informal)

Consider the entire input space $\mathcal{X}$, and we are given a dataset with size n for learning the predictor f from arbitrary $\mathcal{F}_{\text{all}}$.

Arbitrary $\mathcal{F}_{\text{all}}$ means we can memorize every possible labeling (memorize everything seen when being optimized).

Seen Inputs

An unrestricted predictor can memorize their labels and achieve zero empirical risk.

Unseen Inputs

If $n < |\mathcal{X}|/2$, **more than half of $\mathcal{X}$ is unseen**. Their labels remain unconstrained.

 (**very informal and hand-wavy**): to guarantee empirical loss below a certain value, the amount of data must grow with $|\mathcal{X}|$.

Practical Choice 2: Restrict the Function Class

Choose a smaller set $\mathcal{F} \subsetneq \mathcal{F}_{\text{all}}$ and run ERM only over that set.

$$\hat{f} \in \arg \min_{f \in \mathcal{F}} \hat{L}_{\mathcal{D}}(f).$$

The function class $\mathcal{F}$ carries assumptions about the problem.

For the house example, the choice of $\mathcal{F}$ (linear functions) assumes that we expect some linear relations between features like house size and the final prices.

Those assumptions reduce computation and allow predictions to generalize beyond memorized examples.

Two Types of Supervised Learning Tasks

In this course, we focus on two types of supervised learning tasks.

Regression

Predict a numerical value.

Classification

Predict a category.

Regression: Predicting a Stock Price

Regression predicts a **numerical label**.



Features X

Recent prices and trading volume

Label Y

Future stock price, $Y \in \mathbb{R}$

Classification: Cat or Dog

Classification predicts a **category**.



$$f(\mathbf{x}_{\text{cat}}) = \text{cat}$$



$$f(\mathbf{x}_{\text{dog}}) = \text{dog}$$

$\mathbf{X}$ = image pixels, $Y \in \{\text{cat}, \text{dog}\},$

Wrap-up

- The ideal objective compares all functions using true risk $L(f)$.
- Empirical risk $\hat{L}_{\mathcal{D}}(f)$ estimates true risk from data.
- ERM first performs **estimation**, then **optimization**.
- Restricting to $\mathcal{F}$ reduces computation and encodes assumptions that support generalization.

 Next lecture: why an average loss estimates expected loss, and what its randomness means.